University of Glasgow | School of Computing Science

# Assessed Coursework

| | | | | |
|---|---|---|---|---|
| Course Name | Systems Programming | | | |
| Coursework Number | 2a: Concurrent `strace` log analyser | | | |
| Deadline | Time: | 16:30 | Date: | 27th Nov 2025 |
| % Contribution to final course mark | 8% | | | |
| Solo or Group ✓ | Solo | ✓ | Group | |
| Anticipated Hours | 12 | | | |
| Submission Instructions | Described in section 5 below. | | | |
| Please Note: This Coursework cannot be Re-Done | | | | |

## Code of Assessment Rules for Coursework Submission

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty, as set out below.

The primary grade and secondary band awarded for coursework submitted after the published deadline will be calculated as follows:

(i) in respect of work submitted not more than five working days after the deadline

a. the work will be assessed in the usual way;

b. the primary grade and secondary band so determined will then be reduced by two secondary bands for each working day (or part of a working day) the work was submitted late.

(ii) work submitted more than five working days after the deadline will be awarded Grade H.

Penalties for late submission of coursework will not be imposed if a good cause is established for late submission. You should submit documents supporting a good cause via MyCampus.

***Penalty for non-adherence to Submission Instructions is 2 bands***

**You must complete an "Own Work" form via https://webapps.dcs.gla.ac.uk/ETHICS for all coursework**
**UNLESS submitted via Moodle**

# Concurrent `strace` log analyser

## 1. Context

When debugging complex software, programmers often rely on a tool called `strace`, which logs every system call (or "syscall") a program makes. A *system call* is a request for a service from the program to the operating system. For example, a program will use syscalls to open/close a file, read from or write to a file, create a process and execute a new program, allocate or free memory, or exchange data between processes.

These `strace` logs can be enormous, containing millions of lines, and analyzing them can be very slow. Typical log lines look like this:

```
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC)
= 3
read(3, "\177ELF\220\243\2\0\0\0\0\0"..., 832) = 832
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7994bce00000
close(3)                                = 0
```

Look at the first line: it shows the syscall (`openat()`), its arguments, and its result (= 3). A result of -1 would indicate a failure. The last line closes the file (`close()`) returning 0, which indicates success. Some syscalls return 0 on success, a negative value on failure. Other syscalls may return context-specific values (a file descriptor, a memory address, the number of bytes read).

The purpose of this assessed exercise is to build a *concurrent* analyser for these log files. The analyser's job is to read a log file and, for every syscall, calculate and print how many times it appears in the log, and how many times it has failed.

## 2. Specification

Large log files typically contain thousands of system call entries. Processing these logs sequentially can be slow. In this exercise, you will develop a concurrent `strace` log analyser in C++ that is significantly faster than the sequential version.

On Moodle, you are provided with a sequential C++17 analyser, `strace-analyser.cpp`.

The program reads a trace file containing system call log entries (one per line), extracts the syscall name and return value, and prints summary statistics for each syscall (number of calls, number of fails).

The given program can run as follows:

```
./strace-analyser <trace_file> [num_threads]
```

where

- `<trace_file>` is a text file containing syscall log lines in the format: `name(args…) = result`
- `[num_threads]` is optional. If omitted, the program runs with one thread. This sequential analyser will not be affected by this argument.

You will convert the program into a **single-producer / multiple-consumers** concurrent design:

- **Producer** *(main thread/coordinator)*: This thread will read the `<trace_file>` line by line and push each line into a work queue.
- **Consumers** *(worker threads)*: These threads (equal to `[num_threads]`) will pop lines from the work queue, parse them, and update a shared data structure containing the statistics.

Your solution must:

- Employ synchronization primitives for concurrency.
- Avoid busy waiting as a synchronization method.
- Shut down gracefully: when the input ends, the producer must close the queue, and consumer threads must finish processing any remaining work before exiting.

## 3. Design and Implementation

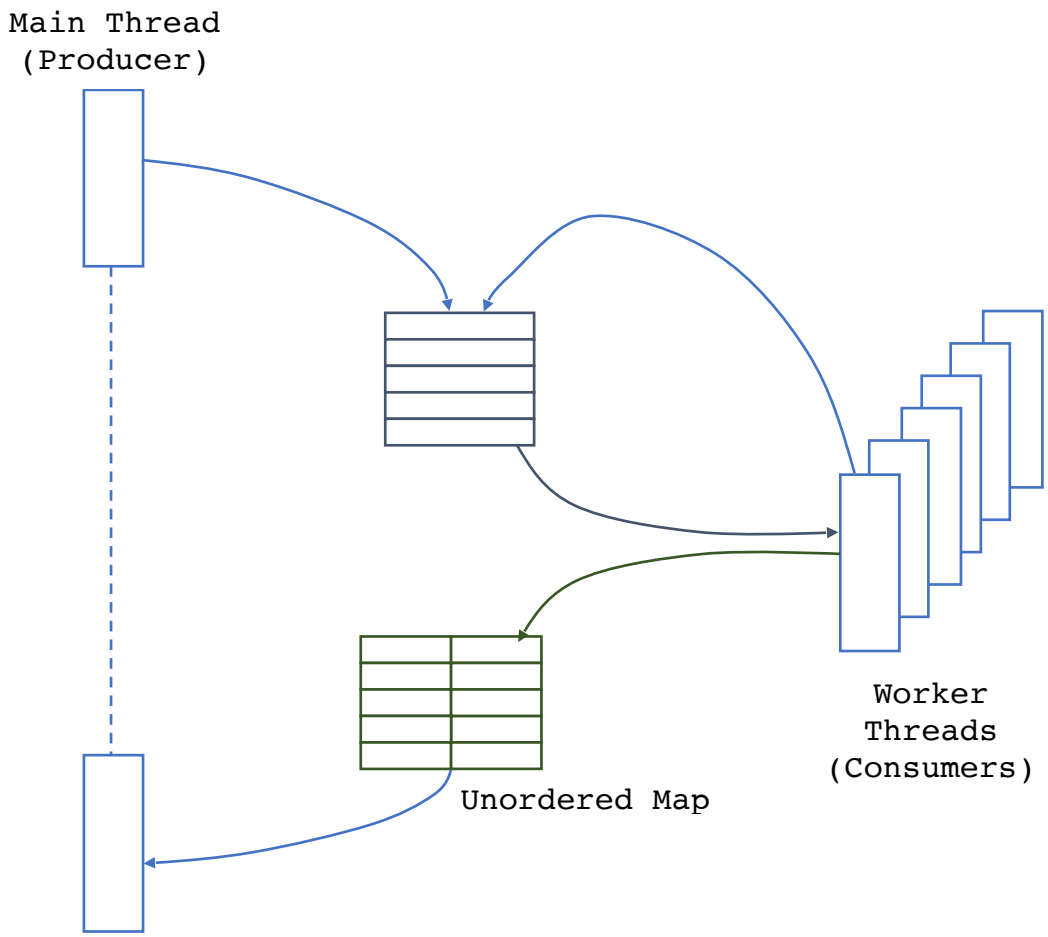
The key data structures, data flows, and threads in the *expected* concurrent version are shown in the figure below. You will implement a **coordinator/workers** concurrency pattern.

The main thread (**coordinator**) reads the trace file line by line and places lines to be processed in the work queue.

The **worker threads** select a line from the work queue, process the line, and add the resulting statistics to a *shared* Unordered Map structure.

You can implement the analyser in **three stages/tiers**, progressing from the provided sequential version to a fully concurrent design. Each tier builds on the previous one and

introduces new concepts in a structured manner. Your final submission should implement **Tier 3**, but you are encouraged to develop and test each tier in order.




### 3.1.1. Tier 1 – Sequential Baseline Using a Work Queue

In this tier, the program remains single-threaded, but you must implement the **Work Queue** and use it to structure the program as if it were concurrent. This introduces the producer–consumer architecture **without using threads**.

a. Implement the Work Queue with push(), pop(), close() functionality

a. Modify `main()` so that the same thread (main thread) performs both the producer step (reads the entire file and pushes lines to the queue), and the consumer step (processes the entire queue).

### 3.1.2. Tier 2 – Single Producer / Single Consumer

In this tier, you introduce **basic concurrency** by separating file reading (producer) and processing (consumer) into **two threads**.

a. Implement a **thread-safe work queue** with appropriate synchronization

b. Modify main() so that the main thread acts as the producer (pushes lines to the queue) and a new, **worker thread** acts as the consumer, processing lines **concurrently to the main thread**.

### 3.1.3. Tier 3 – Single Producer / Multiple Consumers

In this tier, you extend the design to support multiple consumer threads, allowing multiple **concurrent threads** to process large log files.

a. The main thread continues to act as the sole producer.

b. Create multiple new threads (as many as `num_threads`) to act as the consumers. **All threads** (main and worker threads) must work **concurrently**.

## *3.2. How to proceed*

- Build and run the sequential program, `strace-analyser.cpp`, to understand how it works. A sample input file `test.log` is provided as a starting file, together with a `Makefile`.

- Study the code and comments in the provided file. Use the documentation at en.cppreference.com to check that you understand how standard C++ containers are used in `strace-analyser.cpp`.

- Gradually introduce concurrency:

    1. Create a Work Queue that encapsulates the existing `std::queue<std::string>` (Tier 1).

    2. Add one worker thread. Make your Work Queue thread-safe. Run the main thread as the producer and use the worker thread as the consumer (Tier 2). Ensure your program terminates correctly once all work has been processed (no busy waiting).

    3. Extend to multiple worker threads. Ensure your statistics are correct (Tier 3).

- Test your program for:

4. Correctness: Verify that output is identical to the sequential version for the same input.

5. Race conditions: Run your program multiple times with the *same input and thread count*. If output varies across runs, you likely have a race. Use a tool such as clang's `-fsanitize=thread` to detect race conditions early.

6. Deadlocks: Run the program on a large input and with high thread counts (e.g., 8 or 16). If the program hangs or never terminates, suspect a deadlock.

# 4. Submission – Please read carefully!

Submit your solutions electronically on Moodle, into the workshop set for Coursework 2: https://moodle.gla.ac.uk/mod/workshop/view.php?id=5507358

We will be using the same workshop for Coursework 2b – Peer Code Review.

The servers are **NOT** a submission system. Instead, they serve as a unified platform where code from all students will be checked to avoid platform-specific discrepancies.

**Your submission must be ANONYMOUS. There should NOT be anything in your submission title, code or report that can identify you e.g. your name, GUID, or any other personal information. If you intentionally or accidentally identify yourself, you will be penalised by 2 bands!**

**The files you submit are expected to be EXACTLY as follows, including filenames**:

- `strace-analyser.cpp` – the modified source file as described above
- `cw2report.pdf` (in PDF only, contents are outlined below)

## *4.1. Report Contents*

Keep your report concise (<2 pages). Your report should include:

1. **Status:** A brief report outlining the state of your solution, and indicating which tier your have achieved: sequential producer/consumer, two-threaded, multi-threaded. It is important that the report is accurate. For example, it is offensive to report that everything works when it won't even compile.

2. **Sequential (i.e., original) & 2-thread runtimes.** If you have achieved Tier 2 and above, your report should include a screenshot showing:

a. The time to run the provided, sequential analyser on the `test.log` file

b. The time to run your threaded analyser (if you have one) with one thread. You will need to use `time` to obtain understandable times. We are interested in the `real time,` which is the wall clock time to run the program.
Remember to pipe the output to a file to keep the screenshot manageable, e.g.
% time ./strace-analyser test.log >temp
real 0m0.030s
user 0m0.007s
sys 0m0.017s

3. **Runtime with Multiple Threads.** If you have achieved Tier 3, run your program on one of the School's STLinux servers. Perform 3 trial runs with 1, 2, 4, and 8 threads. Record the execution time (`real time`) and the median for the three runs.

| **Threads** | **1** | **2** | **4** | **8** |
|---|---|---|---|---|
| **Execution Time/ Run 1** | | | | |
| **Execution Time/ Run 2** | | | | |
| **Execution Time/ Run 3** | | | | |
| **Median Execution Time** | | | | |

4. **Discussion.** Briefly describe what you conclude from your experiment about the benefits of additional cores for this input data and the variability of elapsed times. You may wish to comment on:
– whether adding more threads improved, or degraded performance,

– how the task size affects scalability,

– the cost of synchronization.

***Note:*** *For this particular workload,* ***it is not expected that the multithreaded implementation will be faster than the sequential version****. In fact, you may observe similar or worse performance when using multiple threads. This is normal and forms an important part of the learning: your discussion should focus on explaining why this occurs rather than on achieving speedup.*

# 5. Marking Scheme

Your submission will be marked on a 100-point scale. Substantial emphasis is placed on **WORKING** submissions, and you will note that a large fraction of the points is reserved for this aspect. It is to your advantage to ensure that whatever you submit compiles, links, and runs correctly.

You must be sure that your code works correctly with `clang++` on the School `stlinux` servers, regardless of which platform you use for development and testing. Leave enough time in your development to fully test on the servers before submission.

The marking scheme is as follows.

| Points | Description |
|---|---|
| **20** | cw2report.pdf<br>**10** if the report accurately & honestly describes the state of your submission<br>**10** if sound experimentation with different numbers of threads and analysis of the results; **0 marks will be awarded** if code is **sequential** (Tier 1), up to **5 marks will be awarded** if code only supports 1 worker thread (Tier 2) |
| **80** | strace-analyser.cpp (submitted)<br>**45** for workable solution (looks like it should work)<br>- 15 for sequential using a work queue (Tier 1)<br>- 35 for single producer/single consumer with one worker thread created and a thread-safe work queue (Tier 2)<br>- 45 for single producer / multiple consumers with multiple worker threads created and a thread-safe work queue (Tier 3)<br>**4** if it successfully compiles with no warnings<br>**5** for correct use of synchronization methods (no busy waiting)<br>**5** for clean and correct thread shutdown<br>**5** if no deadlocks observed<br>**8** if it works correctly (when tested with test.log file)<br>**8** if it works correctly when tested with unseen log file |

The following points should be noted regarding the marking schemes.

- Your report needs to be accurate and honest. Your program should be tested on the School Servers – this is particularly important for this assessment, where we want to observe runtimes with a common baseline for the serial program. The 10 points associated with the report are probably the easiest 10 points you will earn, as long as you are honest.

- The points associated with “workable solution” are the maximum number of points that can be awarded. If it is deemed that only part of the solution appears workable, then a portion of the points in that category will be awarded.